

CSCE 678 DISTRIBUTED SYSTEMS AND CLOUD COMPUTING · MODULE 10 · LECTURE NOTES

Netcode II: Prediction and Reconciliation

Feeling instant while waiting for permission.

MODULE OUTCOMES

- Implement client-side prediction.
- Implement server reconciliation with input replay.
- Implement entity interpolation and state the buffer trade.
- Explain rollback netcode and its requirements.
- Diagnose the characteristic artifacts of each.

1 Client-side prediction

THE CLIENT SIMULATES ITS OWN ACTIONS IMMEDIATELY

Without prediction, pressing forward does nothing at all for a full round trip — the input travels to the server, the server simulates, and the new state travels back. At 100 ms RTT that is a tenth of a second of complete unresponsiveness on every single input, which is unplayable for anything requiring precision.

With prediction, the client applies the input to its own copy of the world immediately, using the same movement code the server will use, and sends the input to the server at the same time.

So movement feels instantaneous, because locally it is. The player's own character responds on the same frame as the keypress, which is the only latency a player directly perceives.

And the client is usually right. It runs the same deterministic movement code against the same input, so in the absence of interaction with other players or server-side events its prediction matches what the server will compute. **The interesting case is when it does not** — someone else shot you, a door closed, a grenade exploded — and §2 is about handling that invisibly.

2 Server reconciliation

CLIENT keeps a buffer of inputs sent but not yet acknowledged, each tagged with a sequence number.

send #42, #43, #44, #45 ... and apply each locally

SERVER replies with authoritative state AND the last input it processed: "here is the world, as of your input #43"

CLIENT then:

1. snap its own state to the server's state
2. DISCARD inputs #43 and earlier from the buffer
3. RE-APPLY #44, #45 ... on top of that state
4. result: a corrected prediction that still contains everything the player has done since

If the prediction was right, step 3 reproduces exactly what the client already had, and NOTHING VISIBLY HAPPENS.

If it was wrong, only the divergence since #43 is lost.

The replay is what makes the correction invisible in the overwhelmingly common case where the prediction was right. **Snapping to the server state without replaying is the classic implementation error:** it discards every input the player has made during the last round trip, so the character jerks backwards on every single update and the controls feel broken. The buffer and the replay are not an optimisation; they are the technique.

SMOOTH THE CORRECTION, DO NOT SNAP IT

A visible snap is jarring, and small corrections happen constantly — from floating-point differences, from timing jitter, from a server tick landing slightly differently. If every one produced a visible jump, the character would shimmer permanently.

So blend toward the corrected position over a few frames rather than teleporting to it. The player sees continuous motion and the error is absorbed.

But large errors must snap. Smoothing a two-metre correction over half a second means the player spends half a second believing they are somewhere they are not — shooting from the wrong position, walking into a wall that is not where it appears. **Being wrong smoothly is worse than being corrected abruptly.**

So threshold it: below roughly 10 cm, blend over a few frames; above it, snap immediately. **And log the large corrections as a metric** — a rising rate of large corrections means either a prediction bug (the client and server movement code have diverged) or a client attempting to move in ways the server rejects, which is Module 11's problem. **The same signal serves debugging and anti-cheat.**

3 Other players

INTERPOLATE REMOTE ENTITIES; DO NOT PREDICT THEM

You know your own input, so you can predict yourself. You do not know anyone else's, so predicting them means *extrapolating* from their last known velocity — which is correct while they move in a straight line and wrong the instant they change direction.

So render remote entities in the past. Buffer incoming snapshots and display the state interpolated between two snapshots you have *already received*, typically 100 ms behind the newest. **You are interpolating between known facts rather than guessing beyond them.**

The result is perfectly smooth motion with no guessing and no correction artifacts — remote players never warp, never snap, and never moonwalk, because nothing was ever predicted.

The cost is that you see everyone else slightly late. You are aiming at where an opponent was 100 ms ago plus your own network delay. **That is exactly the unfairness lag compensation exists to address** (Module 11 §2) — and the fix creates problems of its own, which is the subject of that module.

Technique	Characteristic artifact	How it is perceived
Prediction	Correction after a misprediction.	Rubber-banding — being pulled back to where the server says you were. Usually means genuine network trouble or a server disagreement.
Prediction with no smoothing	A visible snap on every correction, however small.	Constant jitter on your own character; controls feel unreliable.
Prediction with too much smoothing	Slow convergence to the authoritative state.	Floaty, disconnected controls — the character feels like it is being dragged rather than driven.
Interpolation	Remote players are rendered in the past.	Shots that visually connect and are scored as misses — the single most reported netcode complaint, and Module 11's subject.
Extrapolation (dead reckoning)	Wrong whenever the extrapolated entity changes direction.	Warping and snapping back — characters sliding past a corner and then jumping back to it.
Too small an interpolation buffer	The buffer runs dry when a packet is late.	Stuttering remote players — freeze, jump, freeze.

Every one of these is a *correct implementation displaying its trade-off*, not a bug. Recognising which artifact you are looking at tells you which parameter to move, and **that diagnostic skill is most of what netcode tuning actually is** — the implementations are not especially hard and the tuning is where the time goes.

4 Rollback

ROLLBACK NETCODE PREDICTS THE OTHER PLAYER TOO

Assume the remote player's input this frame is the same as their last frame's input, and simulate forward immediately. No waiting for their input to arrive, and no interpolation delay — both characters respond on the current frame.

When the real input arrives and differs from the assumption, roll the simulation back to the frame it applies to and re-simulate every frame since with the correct input. The display jumps to the corrected state.

So both players see an instant response to their own input, and occasional visual corrections of the *other* character — which is the opposite allocation of the error from interpolation.

It requires the simulation to be deterministic, cheap enough to re-run several frames within one frame's budget, and fully snapshot-able so that the rollback point can be restored. **Easy for two fighters on a small stage; infeasible for a large world with hundreds of entities**, which is why the technique is associated with one genre.

WHY ROLLBACK BEATS INPUT DELAY

The alternative is adding input delay — holding your own input for a few frames so the remote input reliably arrives in time to simulate them together. **This makes *both* players' controls less responsive**, permanently, by the amount of the worst connection.

Rollback instead keeps local response instant and makes the remote character occasionally correct itself — moving the entire cost of the network onto the visual representation of the opponent.

Players tolerate the second far better than the first. Your own character feeling sluggish is immediately and continuously noticeable; your opponent's sprite flickering occasionally during a correction is barely registered, and in practice players report rollback connections as 'better' at latencies where delay-based netcode is reported as unplayable.

This is a human-factors result rather than a technical one, and it is the same observation CSCE 650 Module 06 made about virtual reality: **what people detect is the coupling between their own action and the response to it**, far more sensitively than they detect inconsistency in the world around them. **Both fields arrived at the same conclusion independently**, which is reasonable evidence it is a fact about perception rather than about either medium.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Glenn Fiedler — Client-Side Prediction and Server Reconciliation (free)	The §1 and §2 technique , with code and with the smoothing detail of the second callout.

Resource	What it covers for this module
Gabriel Gambetta — Fast-Paced Multiplayer (free, interactive)	Interactive demonstrations of prediction, reconciliation, and interpolation , with sliders for latency. The single best way to build intuition for this module.
Valve — Latency Compensating Methods in Client/Server Protocol Design (free)	The original Half-Life paper. Covers §3's interpolation and sets up Module 11.
Infil — The Fighting Game Glossary, and GGPO's documentation (free)	Rollback explained from both the player's and the implementer's side — the §4 human-factors argument is clearest in the player-facing writing.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Build the no-prediction baseline** and play it at 100 ms. Record how it feels.
- 2 Add client-side prediction and compare.
- 3 **Implement reconciliation with input replay.**
- 4 **Then remove the replay** (snap only) and demonstrate the character jerking backwards on every update.
- 5 Add smoothing with a threshold. Tune the threshold and report what each value feels like.
- 6 **Log correction magnitudes** and plot the distribution at 50, 150, and 300 ms.
- 7 Implement entity interpolation with a 100 ms buffer.
- 8 **Shrink the buffer until remote players stutter** and report the threshold against your jitter.
- 9 Implement extrapolation instead and **demonstrate the warping on direction change.**
- 10 **Implement rollback** for a two-player simulation and measure how many frames you can re-simulate within one frame's budget.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What does client-side prediction do, and why is the client usually right?
- 2 Describe reconciliation's four steps and say why the replay matters.
- 3 What happens if you snap without replaying?
- 4 When should a correction be smoothed and when snapped?
- 5 Why can you predict yourself but not other players?
- 6 What does interpolation cost, and what does it buy?

- 7 Name six artifacts and the technique each comes from.
- 8 Describe rollback and its three requirements.
- 9 Why do players prefer rollback to input delay?